Week 6 Lab Handout

API Build II — Writes, Validation, and Real State Changes

Time: 110 minutes **Bring: laptop, openapi.yaml, Week 5 GET endpoints, Swagger UI open**

Before you start: a wrong write doesn't fix itself

Last week, a wrong response was embarrassing but harmless — fix the bug, the next request is correct, nothing was damaged in between. Writes don't work that way. A `POST` that accepts bad data creates a real, permanent record. A `DELETE` that doesn't actually delete leaves a real row sitting in your database. Fixing the bug tomorrow doesn't undo what already happened today.

> **That's the entire reason today's habit is validate first, write second** — and why "it probably works" is exactly the sentence every write bug starts with.

Part A — Quick Setup (10 min)

1. Confirm your Week 5 GET endpoints still run and still pass their contract verification — today builds directly on top of that server, not a fresh one.
2. Open your `openapi.yaml` and find every endpoint that isn't a GET: your POST, PUT/PATCH, and DELETE routes.
3. For each one, re-read its `requestBody` schema. Note exactly which fields are `required` — this is your validation checklist for Part B, already written for you by the contract.

Part B — Write Your Validation First (25 min)

Before you write any logic that actually changes data, write the checks that reject bad input. This order matters: validation-first means invalid data never has a chance to reach your database in the first place.

For every write endpoint, check:

1. **Are all required fields present?** Your contract already lists them.
2. **Does every field have the right type?** A string where the contract says integer is not close enough to pass.
3. **Is the value actually usable?** An empty string title, a malformed date — technically present, still not valid.

If any check fails, respond immediately with `400 Bad Request` and a clear message — and stop. Nothing gets written until all three checks pass.

```
if (!req.body.title || !req.body.dueDate) {
return res.status(400).json({ message: 'title and dueDate are required' });
}
```

Part C — Implement Your Write Endpoints (35 min)

With validation already rejecting bad requests, implement the actual state change for each endpoint — and return the status code that matches what really happened, not a default 200 for everything.

Outcome	Status code
POST succeeds	201 Created

PUT / PATCH succeeds	200 OK
DELETE succeeds	204 No Content
Request fails validation	400 Bad Request
Updating or deleting something that doesn't exist	404 Not Found

As you build, keep asking

- Did I check the resource actually exists before updating or deleting it?
- Does my POST response include the newly created resource, shaped to match the contract exactly — same mapping-step discipline as Week 5?
- If I call this endpoint twice with the same input, does the result match what Week 3's idempotency rules say it should?

Part D — Verify With Bad Input, Not Just Good Input (25 min)

Last week's habit was “verify, don't eyeball.” This week it gets harder: verifying a write means deliberately trying to break it, not just confirming it works when you feed it exactly what it expects.

For **every** write endpoint, test both directions:

1. **Send a valid request.** Confirm the success status code, and confirm the data actually changed (re-fetch it with your Week 5 GET endpoint to check).
2. **Send an invalid request on purpose** — missing a required field, wrong type, an ID that doesn't exist. Confirm you get 400 or 404, not a silent 200.

What you're specifically checking for

- [ ] Every required field is actually enforced, not just documented
- [ ] Every success case returns the correct status code (not a blanket 200)
- [ ] PUT/PATCH/DELETE on a nonexistent ID returns 404, not a silent success
- [ ] A rejected request doesn't leave partial or garbage data behind
- [ ] Sending the same valid PUT twice produces the same end state, not two different ones

An endpoint you've only ever sent correct input to is not verified — it's demonstrated. Those are not the same thing.

Part E — Finalize (15 min)

If testing today revealed the contract itself needs a change — a field that should be optional, a status code you hadn't planned for — update `openapi.yaml` and add an entry to `CONTRACT_DEVIATIONS.md`, same as last week. If nothing changed, confirm the file still accurately says so.

Commit and push:

```
git add . CONTRACT_DEVIATIONS.md
git commit -m "Week 6: write endpoints implemented with validation"
git push
```

(Still no branch-naming convention to follow — that starts once you're working out of the template repository your instructor hands you.)

This week's deliverable checklist

- [ ] Every write endpoint from openapi.yaml is implemented and running
- [ ] Validation rejects missing or wrong-type fields with 400, before anything is written
- [ ] Every success case returns the correct status code (201/200/204, not a blanket 200)
- [ ] Every write endpoint has been tested with bad input, not just good input
- [ ] PUT/PATCH/DELETE on a nonexistent resource returns 404
- [ ] CONTRACT_DEVIATIONS.md updated if the contract changed, or confirmed unchanged
- [ ] All code committed and pushed

Rubric (40 pts)

Criterion	Points
All write endpoints (POST/PUT/PATCH/DELETE as applicable) implemented and functional	12
Input validation rejects malformed or missing data with correct 400 responses	10
Correct status codes for every case (201/200/204/400/404)	10
Evidence of testing with bad input, not just the happy path	8

Common problems and quick fixes

Problem	Fix
Every response comes back 200, even for POST	Set the status explicitly on every route — most frameworks default to 200 unless you say otherwise
A PUT to a nonexistent ID creates a new record instead of failing	Check for existence first; if nothing is found, return 404 and stop — don't let an update silently become a create
Validation logic works but is copy-pasted into every route	That's fine for today's scope, but worth noticing — a shared validation function is the natural next refactor once all your endpoints work
Not sure what counts as an “invalid” value beyond missing/wrong-type	Ask what your contract's enum, format, or required fields actually promise — anything that breaks that promise is invalid
DELETE returns success but the record is still there on the next GET	Confirm your delete query is actually matching the right ID, and that you're not just returning success without executing anything
Sending the same PUT twice gives two different results	That's an idempotency violation — revisit Week 3's rule: a PUT should set an absolute state, not increment or toggle one

Next week: does your API survive contact with someone else's tests, not just your own? Testing begins.